DatumLex: extração de dados e publicação automática

Data: 24 de outubro de 2026

Escopo: implementação atual e processo de atualização diária.

Objetivo

O pipeline do DatumLex coleta informações judiciais públicas, valida e normaliza os dados, preserva sua origem e armazena os registros processados no PostgreSQL utilizado pela aplicação na Render.

Existem dois modos de execução:

- **Carga inicial ou de ampliação:** um banco local de preparação organiza lotes para publicação transacional na Render e interrompe a coleta na meta definida.
- **Atualização diária:** uma tarefa agendada no Windows executa os conectores e grava no PostgreSQL da Render por conexão TLS autorizada.

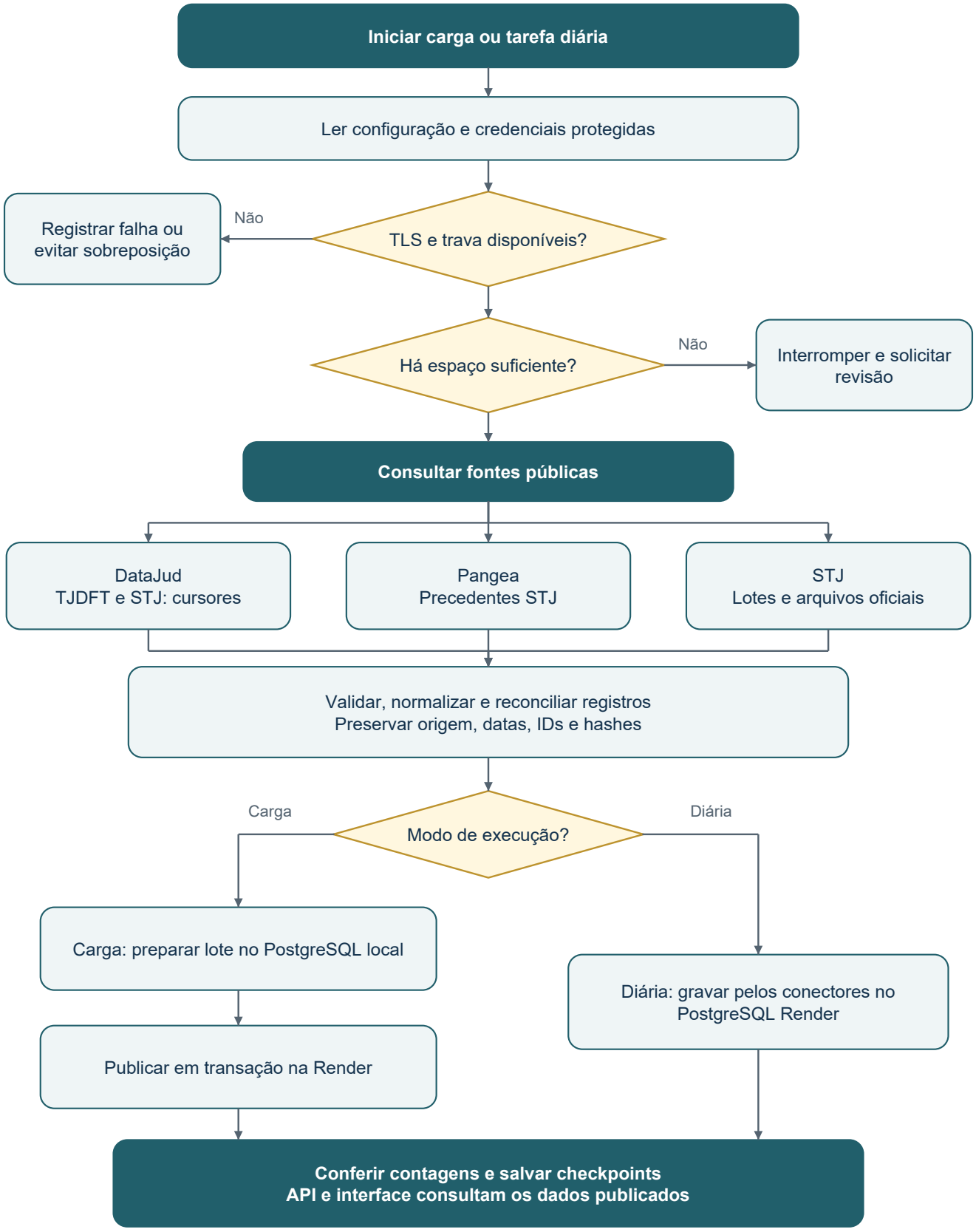
A meta inicial de **10.000 registros armazenados foi atingida**. A tarefa diária está configurada para **19h40, no horário de Brasília**, mas permanece **desativada**, aguardando decisão da equipe para reativação.

Fontes de dados

Fonte	Informações coletadas	Papel no DatumLex
DataJud: TJDFT e STJ	Metadados públicos, assuntos, datas, tribunais e movimentações processuais.	Alimenta o warehouse analítico. Cada fato representa um documento da fonte, não necessariamente um recurso ou processo CNJ único.
Pangea: STJ	Referências de precedentes e teses, com pesquisas de responsabilidade civil, dano moral e dano material.	Alimenta referências jurídicas candidatas. Importar não equivale a aprovar juridicamente.
Dados Abertos do STJ	Metadados de publicações e arquivos de texto oficiais disponíveis.	Fornece proveniência documental e textos, separadamente dos fatos DataJud.

Essas populações possuem significados e campos de data diferentes. Não devem ser combinadas indiscriminadamente no cálculo dos indicadores jurídicos.

Fluxograma geral



Carga: repetir até a meta. Diária: concluir e aguardar o próximo horário.

Os ramos representam etapas lógicas. A rotina diária executa as fontes em sequência: **DataJud, Pangea e publicações do STJ**.

Carga inicial e ampliação

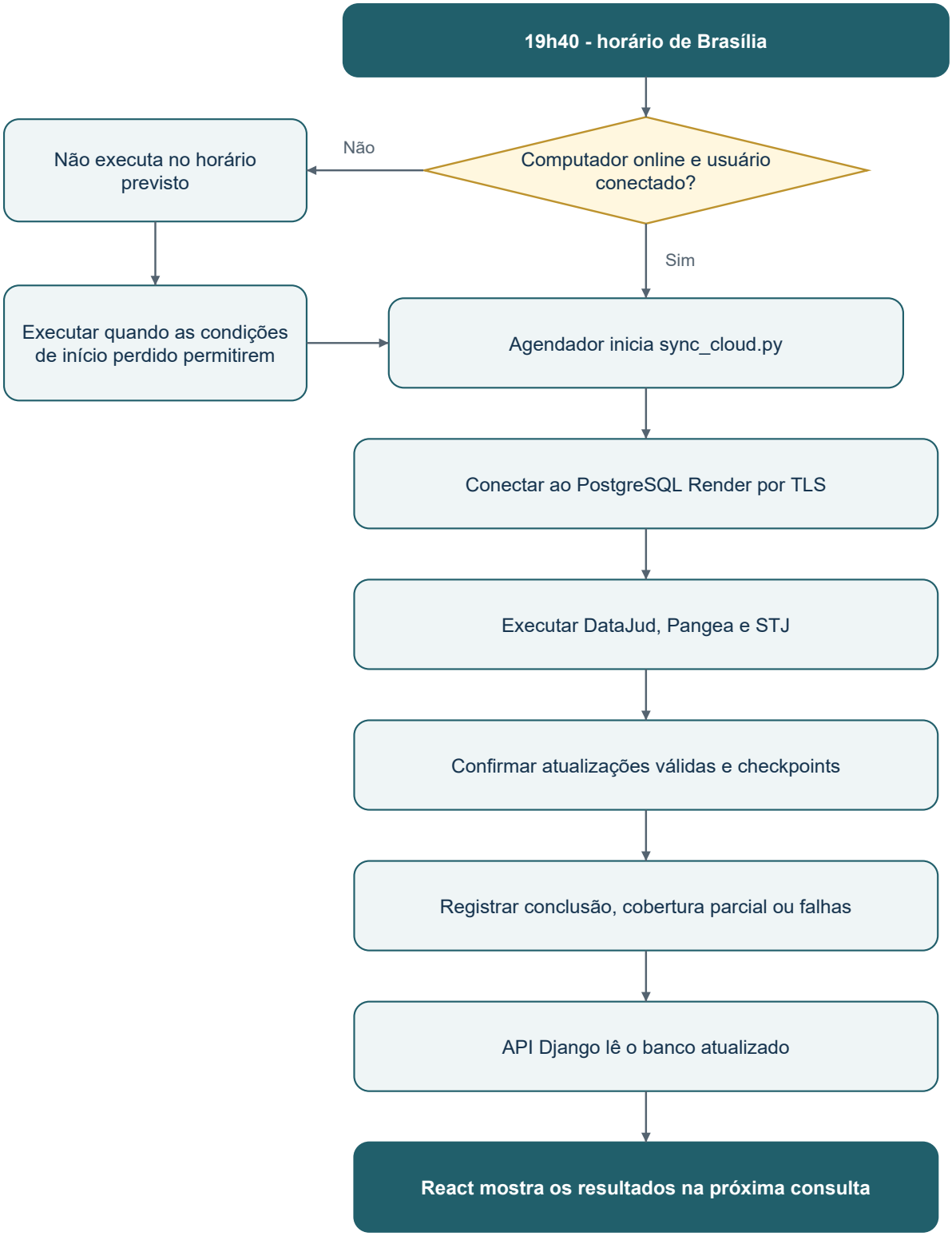
```
APIs públicas -> validação e normalização -> PostgreSQL local de preparação
-> publicação transacional em lotes -> PostgreSQL Render -> API Django -> React
```

1. Adquirir a trava do ETL, impedindo publicação simultânea de outro coletor gerenciado.
2. Criar um banco separado de preparação a partir do warehouse judicial existente e preservar o banco local original.
3. Retomar os cursores confirmados do DataJud, com lotes de até 500 registros. O lote DataJud do STJ foi reduzido a 100 após resposta incompleta do provedor.
4. Reutilizar localmente as regras da aplicação para normalização, deduplicação, classificação de resultados e carga do warehouse.
5. Transferir os snapshots por PostgreSQL `COPY` e reconciliá-los em uma transação remota. Remover movimentações e vínculos de assuntos retirados pelo snapshot correspondente da fonte.
6. Conferir contagens e categorias de resultado, confirmar a publicação e salvar o checkpoint.
7. Repetir até atingir a meta ou encontrar uma condição de segurança que exija interrupção.

O publicador não escreve nas tabelas de contas, senhas, permissões ou sessões. Sua reconciliação foi verificada quanto à republicação, remoção de eventos obsoletos, contagem de linhas e mapeamento semântico dos resultados.

A meta soma fatos DataJud, referências jurídicas e metadados de publicações STJ. Não inclui movimentações, linhas de associação, versões ou logs. **Esse total operacional não representa casos jurídicos únicos entre fontes.**

Extração diária e publicação automática



Configuração existente: tarefa diária atualmente desativada.

Item	Configuração
Tarefa	`DatumLex-Cloud-ETL`
Horário	Diariamente às 19h40, no fuso de Brasília configurado no Windows
Situação atual	Desativada
Ponto de entrada	`D:\Api\datumlex-operations\sync_cloud.py`
Limite DataJud	Até 10 páginas por tribunal; retomadas preservam o tamanho de página salvo
Limite Pangea	Até 5 páginas de 100 resultados por pesquisa
Pesquisas Pangea	`responsabilidade civil`, `dano moral`, `dano material`
Publicações STJ	Busca limitada em lotes recentes disponíveis, evitando datas já processadas sem falhas de schema
Destino	PostgreSQL da Render
Rede	TLS e acesso externo restrito ao IP autorizado do computador
Acompanhamento	Logs locais com rotação e resultados das tarefas do Windows

A tarefa diária não repete indefinidamente a carga inicial. Entretanto, um histórico DataJud parcialmente carregado continua do cursor salvo antes de o comando de sincronização iniciar a atualização incremental. O limite de páginas restringe uma execução; ele não comprova cobertura integral da fonte.

Qualidade e recuperação

- **Deduplicação:** preservar identificadores da fonte e reconciliar registros existentes. Pesquisas diferentes podem retornar a mesma referência Pangea.
- **Datas:** manter distintos o ajuizamento no DataJud, a publicação no STJ e a data de decisão revisada.
- **Proveniência:** preservar URLs, execuções de ingestão, identificadores e hashes de conteúdo disponíveis.
- **Respostas incompletas:** não carregar páginas DataJud identificadas como incompletas. As páginas já confirmadas permanecem disponíveis.
- **Publicação interrompida:** proteger os lotes com transações remotas. Checkpoints e preparação local permitem recuperação.
- **Textos ausentes:** registrar explicitamente os arquivos indisponíveis no ZIP, sem sintetizar conteúdo judicial.
- **Revisão jurídica:** manter referências Pangea pendentes de aprovação. O código operacional preserva a aprovação humana quando o conteúdo da fonte permanece idêntico; conteúdo alterado exige revisão.
- **Acesso:** publicar dados judiciais não substitui contas nem altera a senha de suporte.

O que é automático e o que permanece separado

Processo	Mecanismo	Situação
Deploy da aplicação	Alterações na `main` do GitHub acionam a Render.	Configurado
Publicação da carga inicial	Worker local publica lotes até a meta.	Concluída em 10.000 registros totais
Atualização diária das fontes	Agendador do Windows executa o ETL.	Configurada e desativada
Aprovação jurídica	Revisão humana das referências importadas.	Não automática
Disponibilização dos textos STJ	Exige armazenamento durável acessível e suporte na aplicação.	A coleta local não disponibiliza os arquivos no site automaticamente

Atualizar o GitHub publica código; executar o ETL atualiza dados. A aplicação hospedada consulta o PostgreSQL da Render, não o banco original no Docker local. Alterações arbitrárias naquele banco local não são replicadas automaticamente para a nuvem.

Os textos do STJ permanecem no computador. Seus metadados e sua proveniência estão no PostgreSQL da Render. É necessário um arranjo próprio de armazenamento para permitir download confiável desses arquivos pelo site.

Requisitos operacionais

Para a automação local funcionar, o computador, o Docker quando houver preparação local, a internet e a sessão do usuário Windows precisam estar disponíveis. Uma mudança no IP público exige ajustar a regra autorizada do banco.

O PostgreSQL atual é uma instância gratuita temporária. O painel indica vencimento em **23 de novembro de 2026**. O ETL não prorroga esse prazo nem substitui uma política de backup. As proteções de armazenamento interrompem a coleta em vez de contratar um plano automaticamente.

Por que esse processo se adapta ao DatumLex

O processo reutiliza as regras de dados do Django, separa a coleta do deploy e disponibiliza as alterações publicadas no mesmo banco usado pela API. A preparação local reduz as viagens repetidas ao banco remoto em cargas maiores. Transações, identificadores e checkpoints apoiam publicação consistente e recuperação. A tarefa diária limitada fornece uma alternativa prática de atualização para a implantação acadêmica quando a equipe decidir ativá-la.